



# Architecture and Agility: Married, Divorced, or Just Good Friends?

Frank Buschmann and Kevlin Henney

**DOES AGILE DEVELOPMENT** need architecture? Does architecture need agile development? Is it possible to even answer these questions without a polarizing debate typified more by caricature and entrenched cultural views than by clear definitions and open reasoning—a debate more closely resembling two monologues streaming past each other than a dialogue?

Perhaps rephrasing the question in more general terms offers a better place to start: instead of focusing specifically on agile approaches, we should consider development processes more broadly. And, rather than posing an assumption as a question, we should consider a more open and neutral question: What is the relationship between architecture and process?

## Architecture and Process

*Architecture* is commonly defined as “the structural decomposition of a system, including its decomposition into parts, their connectivity, interaction mechanisms, and the guiding principles that inform the design of the system.”<sup>1</sup> Although not incorrect from a technical perspective, this definition is open to a broad spectrum of interpretation. It can mean anything from a high-level design sketch with little relationship to technology, code, or the actual system being built<sup>2</sup> to a big, rigid up-front design with a lot of class- and code-level minutiae. In practice, neither view offers sufficient guidance for the actual process of development, a necessary contribution to “good” architecture. The first form

is too abstract to give concrete guidance; the second is too premature, decisive before relevant details are actually known. Therefore, it’s no surprise that some in the agile community hold that architecture is not a core concern in practical software development.

Grady Booch<sup>3</sup> and Martin Fowler,<sup>4</sup> in contrast, offer value-oriented definitions of software architecture. Both define architecture in terms of significant decisions that are costly and hard to change. Paul Dyson and Andy Longshaw extend the structural consideration of architecture with a rationale—the “why”—that guides design decisions.<sup>5</sup> These definitions help us see architecture as something that responds to a set of needs, such as functional requirements, operational characteristics, and developer habitability.

In practice, expressing a software architecture can include a cross section of decisions, from a handwritten sketch down to critical details, some of which are properly and explicitly acknowledged as decisions, some of which are assumptions or givens, and some of which are decisions that are made unintentionally and recognized as significant only in hindsight. Consequently, architecture becomes a service to guide and develop a software system toward fulfillment of a set of business and technical objectives,<sup>6</sup> expressed in a form that best helps communication and sharing. It’s not a technical artifact in its own right, expressed as a purely formal description.

*Process* can be considered the answer to a set of questions: Who is doing what, when,



how, why, and for whom? All software development projects, therefore, have a process, even if it's not explicit: Whatever answers these questions can be considered to be the process, whether it's loose or formal, whether it gives a team active control over development or not.

Yet, we can see that how we staff, budget, and schedule a project are decisions that will significantly influence the choice and viability of any architecture. This is true of many things, from the forcing function that Conway's law<sup>7</sup> applies to how a system is partitioned, regardless of its original vision, to the questions of technology choice and skills, intended scope and release model, and—ultimately—the actual system design.

Interdependency among these many influences, coupled with a tendency to change over time, demands processes that establish both clear decision chains for architecture-relevant aspects and direct, meaningful feedback loops to adjust any decision in light of new information, responding to the inevitable consequences of discovery and invention. A process must support a project team, not vice versa—project teams aren't paid for meeting a process but for delivering running software! Agile processes target exactly this goal.

### What Architecture Means for Agile Development

In contrast to some discussions we observe in the software community, agile development is not about cargo cult adoption of Scrum or any other process, toolkit, or methodology—although we certainly observe this and consider it a problem. The essence of agility is responsiveness, learning, and sufficiency. Agility is reflected in sustainability and quality in software and its development—by definition, unsustainable and poor-quality development contradict and reduce agility. *The Manifesto for Agile Software Development* notes that

“continuous attention to technical excellence and good design enhances agility” (<http://agilemanifesto.org/principles.html>), offering architecture a clear role within agile, although not one expressed through big up-front design.

Our friend and colleague Jens Coldewey has commented that architecture is what makes you fast. With all the agile will in the world, an architecture that resists the changes you want to make because it's poorly conceived or misaligned with its purpose will hinder both good process and good architecture. The architecture will play against the development team and the flow of requirements rather than with the team and its growing understanding of the system's purpose and realization.

When it comes to agile, pragmatic architects need to focus on one simple question: What architectural issues block a team's agility? Although a simple question, the technical excellence and design expertise needed to negotiate a terrain of concerns mean the answer is neither simple nor easy. Modularization in conflict with the application domain model, unnecessary coupling, and chatty component interaction all degrade developer understanding and push up build times and delivery times,

### What Architecture Needs from Process

Where does architecture come from? Does the stork bring it? Is it delivered by email? Does it spring fully formed from the head of an all-knowing, all-seeing architect? Does it emerge serendipitously and as if by magic?

Let's pause a moment to consider that architecture is an expression of knowledge. Software development is knowledge work, which entails learning—learning about technologies, learning what the customer wants, learning how to communicate with those who shape product requirements, learning how best to develop a solution from practices to design choices, learning how to work together, and so on. In short, knowledge comes from learning, and learning takes time—which is precisely what a process structures.

The point of least knowledge—of greatest ignorance—is at the start of any development, whether for a whole product or a release. With this in mind, it's both unwise and irresponsible to make all the significant decisions up front. On the other hand, at the end of a project, we have the greatest knowledge but the least opportunity to exercise it. In short, what architecture

Project teams aren't paid for meeting a process but for delivering running software!

discouraging programmers from testing, making small changes, and experimenting. Any form of unnecessary irreversibility in design<sup>4</sup> and reckless, unstructured technical debt is likely to push up costs while pushing down habitability and consequent agility.

wants from process is space and time to learn, which requires that it's not all done and signed off as final at the start, that it can be considered an empirical activity, with each architectural decision seen as a hypothesis to be validated and fed back into the next decision.

Such a response- and learning-based approach also accommodates a flow of requirements. Importantly, even when requirements might appear to be fixed and signed off, there is still a flow of requirements because clarifications and markets and politics cause a steady (or unsteady) stream of change.

**I**n software, as in life, we often understand by trying to classify things and reason about them. And in software, as in life, we often realize as we learn that many of our categories create false and sharp distinctions and dichotomies that obscure what we're trying to understand. So it is with architecture and agility.

What you can build is influenced and constrained by how you build it; how you build something is influenced

and constrained by its significant design features. This close relationship between architecture and process means that architects must think beyond structure and technology to how a system's development unfolds in time. A master-planned approach doesn't best serve the software, its developers, or the organization around them; an agile process responds to the flow of discovery and changes that surround any system. Pragmatic architects need to adopt the role of recognizing and removing any obstacles to the flow that would frustrate developers and stakeholders, causing them to work around the architecture rather than with it. 

## References

1. J. Rumbaugh, I. Jacobsen, and G. Booch, *The Unified Modeling Language Reference Manual*, Addison-Wesley, 1998.
2. J. Spolsky, "Architecture Astronauts Take Over," 1 May 2008; [www.joelonsoftware.com/items/2008/05/01.html](http://www.joelonsoftware.com/items/2008/05/01.html).
3. G. Booch, "On Design," blog, Mar. 2006; [www.ibm.com/developerworks/mydeveloperworks/blogs/gradybooch/entry/on\\_design?lang=en](http://www.ibm.com/developerworks/mydeveloperworks/blogs/gradybooch/entry/on_design?lang=en).
4. M. Fowler, "Who Needs an Architect?," *IEEE Software*, vol. 20, no. 5, 2003, pp. 11–13.
5. P. Dyson and A. Longshaw, *Architecting Enterprise Solutions*, John Wiley & Sons, 2004.
6. R. Faber, "Architects as Service Providers," *IEEE Software*, vol. 27, no. 2, 2010, pp. 33–40.
7. M. Conway, "How Do Committees Invent?," *Datamation*, Apr. 1968, pp. 28–31.

**FRANK BUSCHMANN** is a senior principal engineer at Siemens Corporate Technology, where he's a principal architect in the System Architecture and Platform Department. Contact him at [frank.buschmann@siemens.com](mailto:frank.buschmann@siemens.com).

**KEVLIN HENNEY** is an independent consultant dealing with software architecture, development processes, and programming techniques and practices. Contact him at [kevin@curbralan.com](mailto:kevin@curbralan.com).

## IEEE STC 2013

25th IEEE Software Technology Conference

**8-11 April 2013**

Salt Lake City, Utah, USA

The IEEE Computer Society is pleased to be the new primary sponsor of the IEEE Software Technology Conference, providing a solid basis for continuing this important venue for sharing critical results. The Computer Society has partnered with Utah State University, the long term STC Conference management team, to continue the proud tradition of bringing you excellence in software technology strategies, tools, techniques, and lessons learned that are directly applicable in your own organizations and careers.

*Register today!*

<http://www.sstc-online.org/>

